

ADVANCED ENCRYPTION SUITE

Crypto Toolkit Project — Technical Report

Team Information	
Team Member 1	Name: Hla Essam Abo El-wafa Aly ID: 2406242151
Team Member 2	Name: Farah Hossam Farouk Ramadan Ibrahim ID: 2406247218
Team Member 3	Name: Hamsa Ihab Mohamed Khairy ID: 2406244140
Team Member 4	Name: Menna Emad El-Din Anwar Abdo ID: 2406248838

1. Introduction

This report presents the design, implementation, and evaluation of the "Crypto Toolkit Project", a desktop application developed using Python and the CustomTkinter GUI framework. The application provides a comprehensive set of cryptographic tools through a user-friendly and tabbed graphical interface.

The system is designed to demonstrate core concepts in modern cryptography by integrating four main categories of operations: Symmetric Encryption, Asymmetric Encryption, Hashing, and Data Encoding. Each module is implemented using widely accepted algorithms and follows recommended security practices to ensure both functionality and reliability.

This project aims to provide a practical and interactive environment for applying cryptographic techniques, highlighting both their theoretical foundations and real-world usage.

1.1 Project Objectives

- Provide a user-friendly GUI for performing common cryptographic operations.
- Implement symmetric encryption using AES (GCM mode), DES (CBC), and 3DES (CBC).
- Implement asymmetric encryption using RSA 2048-bit with OAEP padding.
- Support data encoding and decoding via Base64, Hex, and URL encoding.
- Implement secure salted hashing using SHA-256 and SHA-512.
- Ensure robustness through proper key management, error handling, and security practices.

2. Algorithms Used

This section provides a technical explanation of every cryptographic algorithm and technique implemented in the application.

1. AES — Advanced Encryption Standard (GCM Mode)

AES (Advanced Encryption Standard) is a symmetric block cipher standardized by NIST in 2001. It operates on fixed-size 128-bit blocks and supports key sizes of 128, 192, and 256 bits. In this project, AES is implemented using a 256-bit (32-byte) key to achieve a high level of security.

The application uses GCM (Galois/Counter Mode), which is an authenticated encryption mode. This means it provides both confidentiality and integrity by encrypting the data while simultaneously generating an authentication tag. This tag is used during decryption to verify that the data has not been modified.

If any tampering occurs, the authentication process fails, and decryption is rejected. This makes GCM significantly more secure than older modes such as CBC, especially for sensitive or critical data.

2. DES — Data Encryption Standard (CBC Mode)

DES is a symmetric encryption algorithm that works on 64-bit blocks and uses a 56-bit key, it is considered insecure because its key is too small and can be brute-forced.

In this project, DES is included only for learning purposes to show how older encryption algorithms worked.

The implementation uses CBC (Cipher Block Chaining) mode. In this mode, each block of plaintext is combined with the previous ciphertext block before encryption. A random Initialization Vector (IV) is used for the first block to ensure that encrypting the same data multiple times produces different results.

3. 3DES — Triple DES (CBC Mode)

We implemented 3DES (Triple DES) as a more secure alternative to the standard DES algorithm. The process works by running the data through a three-step cycle: Encrypt, Decrypt, and Encrypt again which effectively boosts the key strength to 112 or 168 bits.

To ensure the application is robust, we added strict validation: the system checks that the user provides a key of exactly 16 or 24 bytes, and if the input is incorrect, it displays a clear error message to guide the user. Additionally, for consistent security, we used CBC mode combined with a random Initialization Vector (IV), following the same best practices used in our DES implementation.

4. RSA — Rivest–Shamir–Adleman

For our asymmetric encryption, we chose RSA because it is a reliable, industry standard cryptosystem for secure key exchange. Our application generates 2048-bit key pairs, which provides a high level of security. To ensure the encryption is secure and consistent, we applied standard padding methods provided by the library to protect the data.

Furthermore, we implemented Hybrid Encryption to overcome RSA's data-size limitations. Since RSA is better suited for small amounts of data, we use it effectively: we generate a random 256-bit AES key to encrypt the actual message (using AES-GCM), and then we use RSA to securely 'wrap' or encrypt that AES key.

Key Management

The application allows users to generate a new RSA key pair, as well as save and load keys using .pem files. This makes it possible to reuse the same keys across different sessions instead of generating new ones each time.

This feature reflects how key management works in real-world systems, where securely storing and reusing keys is an essential part of encryption.

5. Hashing — SHA-256 and SHA-512 with Salt

Hashing is a process that converts data into a fixed-size value (called a hash). It is a one-way operation, meaning the original input cannot be recovered from the hash. Even a small change in the input produces a completely different output. In this project, two hashing algorithms are used:

- SHA-256, which produces a 256-bit output (64 hexadecimal characters)
- SHA-512, which produces a 512-bit output (128 hexadecimal characters)

Both algorithms are part of the SHA-2 family and are widely used in modern security systems.

Salted Hashing: To improve security, a salt is added before hashing. A salt is a random value combined with the input so that the same data does not always produce the same hash.

This helps protect against attacks such as rainbow tables. Even if two users use the same password, different salts will result in different hashes.

The application displays the salt along with the hash so it can be stored and reused later for verification.

6. Data Encoding — Base64, Hex, URL

In our toolkit, we included a data encoding module. A key point to highlight is that encoding is not encryption. Unlike encryption, it does not provide any confidentiality and does not require a secret key it is fully reversible by design. Its primary purpose is to convert binary data into a "text-safe" format so that it can be safely stored or transmitted across systems without getting corrupted.

We implemented three standard encoding/decoding methods:

Method	Why we use it	Where we use it
Base64	Converts binary data into a text safe string. This stops data from getting corrupted when sent over text only systems.	Sending data through APIs, handling image/file attachments.
Hex	Turns every byte into two simple characters. It makes raw data "readable" to us humans.	Debugging code and checking what's actually inside the data.
URL Encoding	Fixes "special characters" in links. It replaces them with code so web browsers don't get confused.	Web forms, search queries, and making URLs work correctly.

3. Code Explanation

This section walks through every major component of the source code, explaining the purpose and logic of each function and block.

Import Statements

```
import customtkinter as ctk
from tkinter import messagebox, filedialog
import base64
import urllib.parse
import os
import secrets

from Crypto.Cipher import AES, DES, DES3, PKCS1_OAEP
from Crypto.PublicKey import RSA
from Crypto.Util.Padding import pad, unpad
from Crypto.Hash import SHA256, SHA512
```

The application begins by importing the required libraries and modules needed for the graphical interface and cryptographic operations.

- **customtkinter (ctk):** Used to build the graphical user interface. It provides a modern and styled version of the standard Tkinter library.
- **tkinter.messagebox and tkinter.filedialog:** Used for user interaction. MessageBox displays alerts and error messages, while FileDialog allows users to save and load files (such as RSA keys).
- **base64:** Used to encode binary data into text format, making it easier to display and store encrypted outputs.
- **urllib.parse:** Provides functions for URL encoding and decoding.
- **os:** Used for system-level operations, such as generating random bytes for Initialization Vectors (IV).
- **secrets:** Used to generate secure random values, such as salts for hashing.
- **Crypto.Cipher (AES, DES, DES3, PKCS1_OAEP):** Provides implementations of encryption algorithms: AES for modern symmetric encryption, DES and 3DES for legacy encryption, and PKCS1_OAEP for secure RSA encryption.
- **Crypto.PublicKey.RSA:** Used to generate and manage RSA key pairs.
- **Crypto.Util.Padding (pad, unpad):** Used to ensure that data fits the required block size for block ciphers like DES and 3DES.
- **Crypto.Hash (SHA256, SHA512):** Used to generate secure hash values for input data.

GUI Setup and Application Structure

```

ctk.set_appearance_mode("dark")
ctk.set_default_color_theme("blue")

class UltimateCryptoToolkit(ctk.CTk):
    def __init__(self):
        super().__init__()
        self.title("Crypto Toolkit Project")
        self.geometry("1050x850")

        self.rsa_key = None

        self.tabview = ctk.CTkTabview(self, width=1000, height=800)
        self.tabview.pack(padx=20, pady=20)

        self.tab_sym = self.tabview.add("Symmetric")
        self.tab_asym = self.tabview.add("RSA")
        self.tab_hash = self.tabview.add("Hashing")
        self.tab_encode = self.tabview.add("Encoding")

        self.setup_symmetric()
        self.setup_asymmetric()
        self.setup_hashing()
        self.setup_encoding()

```

The application uses CustomTkinter to create a modern graphical interface. The appearance is set to dark mode with a blue theme.

The main class "UltimateCryptoToolkit" initializes the application window, sets its title and size, and defines a variable "rsa_key" to store RSA keys during runtime.

A tabbed interface (CTkTabview) is used to organize the application into four sections: Symmetric, RSA, Hashing, and Encoding.

Each tab is initialized using separate setup functions, which helps keep the code organized and easy to maintain.

Copy to Clipboard Function (copy_val)

```

def copy_val(self, widget):
    try:
        data = widget.get("1.0", "end-1c")
        if not data:
            messagebox.showwarning("Warning", "Nothing to copy!")
            return
        try:
            import pyperclip
            pyperclip.copy(data)
        except Exception:
            self.clipboard_clear()
            self.clipboard_append(data)
            messagebox.showinfo("Success", "Copied to clipboard!")
    except Exception as e:
        messagebox.showerror("Error", str(e))

```

This function copies text from a textbox to the system clipboard.

It first checks if the textbox is empty and shows a warning if so.

It tries to use the pyperclip library for copying. If it is not available, it uses Tkinter's built-in clipboard methods as a fallback.

A success message is shown after copying, and any errors are handled using exception handling.

Key Preparation Function (make_key)

```
@staticmethod
def make_key(raw: bytes, size: int) -> bytes:
    """Pad or truncate raw bytes to exactly `size` bytes using null-byte padding."""
    return raw.ljust(size, b'\0')[:size]
```

This function adjusts the input key to a required fixed size. It either pads the key with null bytes if it is too short or truncates it if it is too long.

This ensures that the key length matches the requirements of encryption algorithms such as AES, DES, and 3DES.

Symmetric Encryption Interface (setup_symmetric)

```
def setup_symmetric(self):
    ctk.CTkLabel(
        self.tab_sym,
        text="Symmetric Encryption (AES-GCM / DES / 3DES)",
        font=("Arial", 16, "bold")
    ).pack(pady=10)

    self.sym_in = ctk.CTkTextbox(self.tab_sym, height=100, width=750)
    self.sym_in.pack(pady=5)

    key_row = ctk.CTkFrame(self.tab_sym, fg_color="transparent")
    key_row.pack(pady=5)

    self.sym_key = ctk.CTkEntry(
        key_row, width=460,
        placeholder_text="Enter Secret Key",
        show="*"
    )
    self.sym_key.grid(row=0, column=0, padx=(0, 8))
```

```
self.sym_key_visible = False
self.sym_key_toggle_btn = ctk.CTkButton(
    key_row, text="Show", width=80,
    command=self.toggle_sym_key
)
self.sym_key_toggle_btn.grid(row=0, column=1)

self.sym_algo = ctk.CTkOptionMenu(
    self.tab_sym, values=["AES (GCM)", "DES (CBC)", "3DES (CBC)"]
)
self.sym_algo.pack(pady=10)

frame = ctk.CTkFrame(self.tab_sym)
frame.pack(pady=10)
ctk.CTkButton(
    frame, text="Encrypt", fg_color="#28a745",
    command=lambda: self.process_sym("enc")
).grid(row=0, column=0, padx=10)
ctk.CTkButton(
    frame, text="Decrypt", fg_color="#dc3545",
    command=lambda: self.process_sym("dec")
).grid(row=0, column=1, padx=10)
```

```

self.sym_out = ctk.CTkTextbox(self.tab_sym, height=120, width=750)
self.sym_out.pack(pady=10)
ctk.CTkButton(
    self.tab_sym, text="Copy Result",
    command=lambda: self.copy_val(self.sym_out)
).pack()

```

This function creates the user interface for symmetric encryption. It includes:

- A textbox for entering the input text.

- A password-style input field for the secret key, with a show/hide toggle button.

- A dropdown menu to select the encryption algorithm (AES-GCM, DES-CBC, or 3DES-CBC).

- Two buttons for encryption and decryption, which call the "process_sym" function.

- An output textbox to display the result.

- A button to copy the result to the clipboard.

This layout allows users to easily perform encryption and decryption using different symmetric algorithms.

Toggle Key Visibility (toggle_sym_key)

```

def toggle_sym_key(self):
    self.sym_key_visible = not self.sym_key_visible
    self.sym_key.configure(show="" if self.sym_key_visible else "*")
    self.sym_key_toggle_btn.configure(
        text="Hide" if self.sym_key_visible else "Show"
    )

```

This function controls the visibility of the secret key input field.

When triggered, it switches between hiding and showing the key. If the key is hidden, it is displayed as asterisks (*). If shown, the actual text becomes visible.

The button text is also updated dynamically between "Show" and "Hide" to reflect the current state.

Symmetric Processing Function (process_sym)

```

def process_sym(self, mode):
    try:
        algo    = self.sym_algo.get()
        raw_key = self.sym_key.get().encode()
        text    = self.sym_in.get("1.0", "end-1c")

        if not raw_key or not text:
            raise ValueError("Key and Text are required!")

```

This function handles both encryption and decryption for symmetric algorithms (AES, DES, and 3DES). It first reads the selected algorithm, the user-provided key, and the input text. It then validates that both the key and text are not empty.

Based on the selected algorithm, the function applies the appropriate encryption or decryption method. AES is used in GCM mode, while DES and 3DES are used in CBC mode.

The result (either ciphertext or decrypted text) is displayed in the output textbox. Any errors during the process are handled using exception handling and shown to the user

AES Encryption and Decryption (GCM Mode)

```
# — AES GCM —
if "AES" in algo:
    key = self.make_key(raw_key, 32)
    if mode == "enc":
        cipher = AES.new(key, AES.MODE_GCM)
        ciphertext, tag = cipher.encrypt_and_digest(text.encode())
        result = base64.b64encode(
            cipher.nonce + tag + ciphertext
        ).decode()
    else:
        raw = base64.b64decode(text)
        nonce = raw[:16]
        tag = raw[16:32]
        ct = raw[32:]
        cipher = AES.new(key, AES.MODE_GCM, nonce=nonce)
        result = cipher.decrypt_and_verify(ct, tag).decode()
```

When AES is selected, the key is first adjusted to 32 bytes (256-bit).

For encryption: A new AES cipher is created in GCM mode. The plaintext is encrypted, producing both ciphertext and an authentication tag. The nonce, tag, and ciphertext are then combined and encoded using Base64 for easy storage and display.

For decryption: The input is Base64-decoded and split into nonce, tag, and ciphertext. The cipher is recreated using the same nonce, and the data is decrypted and verified using the authentication tag. If verification fails, decryption is rejected.

DES / 3DES Encryption Logic (within process_sym)

```
# — DES / 3DES CBC —
elif "DES" in algo:
    if "3DES" in algo:
        key_len = len(raw_key)
        if key_len not in (16, 24):
            raise ValueError(
                f"3DES key must be exactly 16 or 24 bytes.\n"
                f"Your key is {key_len} byte(s).\n\n"
                f" 16 bytes = 112-bit effective security\n"
                f" 24 bytes = 168-bit effective security\n\n"
                f"Please enter a key of exactly 16 or 24 characters."
            )
        key = raw_key
        cipher_mod = DES3
        blk = 8
```

```
else:
    key = self.make_key(raw_key, 8)
    cipher_mod = DES
    blk = 8

    if mode == "enc":
        iv = os.urandom(8)
        cipher = cipher_mod.new(key, cipher_mod.MODE_CBC, iv)
        ct = cipher.encrypt(pad(text.encode(), blk))
        result = base64.b64encode(iv + ct).decode()
    else:
        raw = base64.b64decode(text)
        iv = raw[:8]
        ct = raw[8:]
        cipher = cipher_mod.new(key, cipher_mod.MODE_CBC, iv)
        result = unpad(cipher.decrypt(ct), blk).decode()
```

```

    else:
        raise ValueError("Unknown algorithm selected.")

    self.sym_out.delete("1.0", "end")
    self.sym_out.insert("1.0", result)

except Exception as e:
    messagebox.showerror("Error", f"Operation failed: {str(e)}")

```

This part of the function handles encryption and decryption using DES and 3DES in CBC mode.

For 3DES, the key length is validated to ensure it is either 16 or 24 bytes, as required by the algorithm. If the key length is invalid, an error message is displayed.

For DES, the key is adjusted to 8 bytes using a helper function.

During encryption:

- A random Initialization Vector (IV) is generated.
- The plaintext is padded to match the block size.
- The data is encrypted using CBC mode.
- The IV is combined with the ciphertext and encoded using Base64 for safe display.

During decryption:

- The input is decoded from Base64.
- The IV is extracted from the beginning of the data.
- The ciphertext is decrypted.
- Padding is removed to recover the original plaintext.

Errors are handled using exception handling to ensure the application does not crash.

Asymmetric Encryption Interface (setup_asymmetric)

```

def setup_asymmetric(self):
    ctk.CTkLabel(
        self.tab_asym,
        text="Asymmetric RSA (2048-bit PKCS1 OAEP)",
        font=("Arial", 16, "bold")
    ).pack(pady=10)

    key_frame = ctk.CTkFrame(self.tab_asym)
    key_frame.pack(pady=5)
    ctk.CTkButton(
        key_frame, text="Generate Key Pair",
        command=self.gen_rsa
    ).grid(row=0, column=0, padx=8)

    ctk.CTkButton(
        key_frame, text="Save Keys to File",
        command=self.save_rsa_keys
    ).grid(row=0, column=1, padx=8)
    ctk.CTkButton(
        key_frame, text="Load Keys from File",
        command=self.load_rsa_keys
    ).grid(row=0, column=2, padx=8)

    self.rsa_in = ctk.CTkTextbox(self.tab_asym, height=100, width=750)
    self.rsa_in.pack(pady=5)

```

```

enc_frame = ctk.CTkFrame(self.tab_asym)
enc_frame.pack(pady=10)
ctk.CTkButton(
    enc_frame, text="Encrypt with Public",
    command=self.rsa_enc
).grid(row=0, column=0, padx=10)
ctk.CTkButton(
    enc_frame, text="Decrypt with Private",
    command=self.rsa_dec
).grid(row=0, column=1, padx=10)

self.rsa_out = ctk.CTkTextbox(self.tab_asym, height=180, width=750)
self.rsa_out.pack(pady=10)
ctk.CTkButton(
    self.tab_asym, text="Copy Keys / Result",
    command=lambda: self.copy_val(self.rsa_out)
).pack()

```

This function creates the user interface for RSA-based asymmetric encryption.

It provides:

- A button to generate a new RSA key pair.
- Options to save and load keys from a file for persistence.
- An input textbox for entering data to encrypt or decrypt.
- Buttons to perform encryption using the public key and decryption using the private key.
- An output textbox to display encrypted data, decrypted text, or generated keys.
- A copy button for easy access to the output.

This interface enables secure key management and allows users to perform RSA encryption and decryption in a simple and organized way. RSA Interface (Encryption / Decryption Controls)

RSA Key Generation (gen_rsa)

```

def gen_rsa(self):
    self.rsa_key = RSA.generate(2048)
    pub = self.rsa_key.publickey().export_key().decode()
    priv = self.rsa_key.export_key().decode()
    self.rsa_out.delete("1.0", "end")
    self.rsa_out.insert("1.0", f"--- PUBLIC KEY ---\n{pub}\n\n--- PRIVATE KEY ---\n{priv}")
    messagebox.showinfo("Success", "RSA 2048-bit Key Pair Generated!")

```

The function "gen_rsa" is responsible for generating a new RSA key pair.

It creates a 2048-bit RSA key, which is considered secure for most applications. The function then extracts both the public key and the private key and converts them into a readable text format.

The keys are displayed in the output textbox, allowing the user to copy or save them. Additionally, a success message is shown to confirm that the keys have been generated successfully.

The generated key pair is stored in the variable "rsa_key" for later use in encryption and decryption.

Saving RSA Keys (save_rsa_keys)

```
def save_rsa_keys(self):
    if not self.rsa_key:
        messagebox.showwarning("Warning", "No keys to save! Generate keys first.")
        return
    path = filedialog.asksaveasfilename(
        defaultextension=".pem",
        filetypes=[("PEM files", "*.pem"), ("All files", "*.*)"],
        title="Save RSA Keys"
    )
    if not path:
        return
    try:
        pub = self.rsa_key.publickey().export_key().decode()
        priv = self.rsa_key.export_key().decode()
        with open(path, "w") as f:
            f.write(f"--- PUBLIC KEY ---\n{pub}\n\n--- PRIVATE KEY ---\n{priv}\n")
        messagebox.showinfo("Success", f"Keys saved to:\n{path}")
    except Exception as e:
        messagebox.showerror("Error", f"Failed to save keys:\n{str(e)}")
```

This function allows the user to save the generated RSA key pair to a file.

First, it checks whether a key pair exists. If not, a warning message is displayed.

A file dialog is then opened to let the user choose the save location and file name (.pem format).

The public and private keys are exported, converted to text format, and written into the file. Both keys are stored together for convenience. Error handling is included to manage any issues during the saving process.

Load RSA Keys (load_rsa_keys)

```
def load_rsa_keys(self):
    path = filedialog.askopenfilename(
        filetypes=[("PEM files", "*.pem"), ("All files", "*.*)"],
        title="Load RSA Keys"
    )
    if not path:
        return
    try:
        with open(path, "r") as f:
            content = f.read()

        if "PRIVATE KEY" in content:
            priv_start = content.find("-----BEGIN RSA PRIVATE KEY-----")
            if priv_start == -1:
                priv_start = content.find("-----BEGIN PRIVATE KEY-----")
            priv_pem = content[priv_start:]
            self.rsa_key = RSA.import_key(priv_pem.strip())
            self.rsa_out.delete("1.0", "end")
            self.rsa_out.insert("1.0", content)
            messagebox.showinfo("Success", "RSA Keys loaded successfully!")
        else:
            messagebox.showerror("Error", "No private key found in file.")
    except Exception as e:
        messagebox.showerror("Error", f"Failed to load keys:\n{str(e)}")
```

The function "load_rsa_keys" allows the user to load RSA keys from a file.

It opens a file dialog for selecting a .pem file, then reads its content. The function searches for a private key inside the file, since the private key contains all the information needed for both encryption and decryption.

If a valid private key is found, it is imported and stored in the "rsa_key" variable. The file content is also displayed in the output textbox for the user.

If no private key is found or an error occurs, an appropriate error message is shown.

RSA Encryption (Hybrid Encryption) — rsa_enc

```
def rsa_enc(self):
    try:
        if not self.rsa_key:
            raise ValueError("Generate or load RSA keys first!")
        from Crypto.Cipher import AES
        from Crypto.Random import get_random_bytes
        text = self.rsa_in.get("1.0", "end-1c").strip()
        if not text:
            raise ValueError("Input is empty!")
        aes_key = get_random_bytes(32)
        cipher_aes = AES.new(aes_key, AES.MODE_GCM)
        ciphertext, tag = cipher_aes.encrypt_and_digest(text.encode())
        cipher_rsa = PKCS1_OAEP.new(self.rsa_key.publickey())
        enc_key = cipher_rsa.encrypt(aes_key)
        result = base64.b64encode(
            enc_key + cipher_aes.nonce + tag + ciphertext
        ).decode()
        self.rsa_out.delete("1.0", "end")
        self.rsa_out.insert("1.0", result)
    except Exception as e:
        messagebox.showerror("Error", str(e))
```

This function performs encryption using a hybrid approach that combines AES and RSA. First, it checks that an RSA key pair is available and that the input text is not empty. The encryption process is performed in four steps:

1. A random 256-bit AES key is generated.
2. The input data is encrypted using AES in GCM mode, producing ciphertext and an authentication tag.
3. The AES key is encrypted using the RSA public key (PKCS1-OAEP).
4. All components (encrypted AES key, nonce, tag, and ciphertext) are combined and encoded using Base64.

This approach improves performance and security, since RSA alone is not efficient for encrypting large data. AES handles the data encryption, while RSA securely protects the AES key.

RSA Decryption (rsa_dec)

```
def rsa_dec(self):
    try:
        if not self.rsa_key:
            raise ValueError("Load or generate RSA keys first!")
        raw = base64.b64decode(self.rsa_in.get("1.0", "end-1c"))
        enc_key = raw[:256]
        nonce = raw[256:272]
        tag = raw[272:288]
        ct = raw[288:]
        cipher_rsa = PKCS1_OAEP.new(self.rsa_key)
        aes_key = cipher_rsa.decrypt(enc_key)
        cipher_aes = AES.new(aes_key, AES.MODE_GCM, nonce=nonce)
        text = cipher_aes.decrypt_and_verify(ct, tag).decode()
        self.rsa_out.delete("1.0", "end")
        self.rsa_out.insert("1.0", text)
    except Exception as e:
        messagebox.showerror("Error", "Decryption failed!")
```

The function "rsa_dec" decrypts data that was encrypted using a hybrid system (RSA + AES-GCM). It first checks if the RSA private key is loaded in "rsa_key". If not, it shows an error message.

Then it reads the encrypted input, decodes it from Base64, and splits it into parts: the RSA-encrypted AES key (first 256 bytes), the nonce, the authentication tag, and the ciphertext.

The AES key is decrypted using RSA, then used to decrypt the ciphertext with AES-GCM while verifying its integrity.

Finally, the decrypted text is displayed in the output box. If anything goes wrong, an error message is shown.

Hashing Interface (setup_hashing)

```
def setup_hashing(self):
    ctk.CTkLabel(
        self.tab_hash,
        text="Secure Salted Hashing",
        font=("Arial", 16, "bold")
    ).pack(pady=10)
    self.h_in = ctk.CTkEntry(
        self.tab_hash, width=600, placeholder_text="Text to hash"
    )
    self.h_in.pack(pady=5)
    salt_row = ctk.CTkFrame(self.tab_hash, fg_color="transparent")
    salt_row.pack(pady=5)
    self.h_salt = ctk.CTkEntry(
        salt_row, width=380,
        placeholder_text="Enter Salt (Highly Recommended)",
        show="*"
    )
    self.h_salt.grid(row=0, column=0, padx=(0, 8))
    self.h_salt_visible = False
    self.h_salt_toggle_btn = ctk.CTkButton(
        salt_row, text="Show", width=80,
        command=self.toggle_h_salt
    )
```

```

self.h_salt_toggle_btn.grid(row=0, column=1, padx=4)
ctk.CTkButton(
    salt_row, text="Generate", width=80,
    command=self.generate_random_salt
).grid(row=0, column=2, padx=4)

self.h_algo = ctk.CTkOptionMenu(
    self.tab_hash, values=["SHA-256", "SHA-512"]
)
self.h_algo.pack(pady=10)

ctk.CTkButton(
    self.tab_hash, text="Generate Hash", command=self.run_hash
).pack(pady=10)

self.h_out = ctk.CTkTextbox(self.tab_hash, height=120, width=750)
self.h_out.pack(pady=10)
ctk.CTkButton(
    self.tab_hash, text="Copy Hash",
    command=lambda: self.copy_val(self.h_out)
).pack()

```

This function creates the user interface for hashing operations.

It includes:

- An input field for entering the text to be hashed.
- A salt input field (hidden by default) with options to show/hide or generate a random salt.
- A dropdown menu to select the hashing algorithm (SHA-256 or SHA-512).
- A button to generate the hash using the selected algorithm.
- An output textbox to display the result.
- A button to copy the hash to the clipboard.

This interface allows users to easily generate secure salted hashes in a simple and organized way.

Toggle Salt Visibility (toggle_h_salt)

```

def toggle_h_salt(self):
    self.h_salt_visible = not self.h_salt_visible
    self.h_salt.configure(show="" if self.h_salt_visible else "*")
    self.h_salt_toggle_btn.configure(
        text="Hide" if self.h_salt_visible else "Show"
    )

```

The function "toggle_h_salt" is responsible for showing or hiding the salt value in the input field.

It first switches the state of "h_salt_visible" between True and False. Based on this value, the function updates the input field to either display the actual salt (when visible) or mask it using "*".

It also updates the button text between "Show" and "Hide" to reflect the current state.

This allows the user to easily control whether the salt is visible or hidden.

Generate Random Salt (generate_random_salt)

```
def generate_random_salt(self):
    random_salt = secrets.token_hex(16)
    self.h_salt.delete(0, 'end')
    self.h_salt.insert(0, random_salt)
```

The function "generate_random_salt" is responsible for creating a random salt value.

It generates 16 random bytes using the `secrets` module and converts them into a hexadecimal string format.

Then, the function clears the salt input field and inserts the newly generated salt value into it.

This allows the user to quickly generate a secure random salt for use in hashing or encryption.

Hash Generation Logic (run_hash)

```
def run_hash(self):
    try:
        text = self.h_in.get().strip()
        if not text:
            raise ValueError("Input text is empty.")
        salt = self.h_salt.get()
        data = (salt + text).encode()
        if self.h_algo.get() == "SHA-256":
            hash_val = SHA256.new(data).hexdigest()
        else:
            hash_val = SHA512.new(data).hexdigest()
        display = (
            f"Algorithm : {self.h_algo.get()}\n"
            f"Salt       : {salt if salt else '(none)'}\n"
            f"Hash        : {hash_val}"
        )

        self.h_out.delete("1.0", "end")
        self.h_out.insert("1.0", display)

    except Exception as e:
        messagebox.showerror("Error", str(e))
```

This function generates a hash value for the input text using the selected algorithm.

First, it checks that the input is not empty. Then, it retrieves the optional salt value and combines it with the input before hashing.

Depending on the selected option, the function applies either SHA-256 or SHA-512 to produce the hash.

The result is displayed along with the algorithm used and the salt value, making it easy to store and reuse for future verification.

Error handling is included to manage invalid inputs or unexpected issues.

Encoding/Decoding Setup (setup_encoding)

```
def setup_encoding(self):
    ctk.CTkLabel(
        self.tab_encode,
        text="Data Encoding / Decoding",
        font=("Arial", 16, "bold")
    ).pack(pady=10)
    self.e_in = ctk.CTkTextbox(self.tab_encode, height=100, width=750)
    self.e_in.pack(pady=5)
    self.e_type = ctk.CTkOptionMenu(
        self.tab_encode, values=["Base64", "Hex", "URL"]
    )
    self.e_type.pack(pady=10)
    frame = ctk.CTkFrame(self.tab_encode)
    frame.pack(pady=10)
    ctk.CTkButton(
        frame, text="Encode", command=self.encode
    ).grid(row=0, column=0, padx=10)
    ctk.CTkButton(
        frame, text="Decode", command=self.decode
    ).grid(row=0, column=1, padx=10)
    self.e_out = ctk.CTkTextbox(self.tab_encode, height=120, width=750)
    self.e_out.pack(pady=10)
    ctk.CTkButton(
        self.tab_encode, text="Copy Output",
        command=lambda: self.copy_val(self.e_out)
    ).pack()
```

The function "setup_encoding" is responsible for creating the user interface for data encoding and decoding.

It starts by adding a title label, followed by a textbox where the user can enter the input data. Then, it provides an option menu that allows the user to choose the encoding type (Base64, Hex, or URL).

Two buttons, "Encode" and "Decode", are added to perform the selected operation.

Below that, an output textbox is created to display the result, along with a **"Copy Output"** button that allows the user to easily copy the processed data.

This setup gives the user a simple interface to encode or decode data in different formats.

Encoding Function (encode)

```
def encode(self):
    try:
        t = self.e_in.get("1.0", "end-1c")
        if not t:
            messagebox.showwarning("Warning", "Input is empty!")
            return
        enc_type = self.e_type.get()
        if enc_type == "Base64":
            res = base64.b64encode(t.encode()).decode()
        elif enc_type == "Hex":
            res = t.encode().hex()
        else:
            # URL
            res = urllib.parse.quote(t, safe='')
        self.e_out.delete("1.0", "end")
        self.e_out.insert("1.0", res)
    except Exception as e:
        messagebox.showerror("Error", str(e))
```

This function converts input data into a selected encoding format.

First, it checks if the input is not empty. Then, based on the selected option, it applies one of the following:

- Base64: converts the data into a Base64 encoded string.
- Hex: converts the data into its hexadecimal representation.
- URL: encodes the data to make it safe for use in URLs.

The encoded result is then displayed in the output textbox.

Error handling is included to manage unexpected issues during the process.

Decoding Function (decode)

```
def decode(self):
    try:
        t = self.e_in.get("1.0", "end-1c")
        if not t:
            messagebox.showwarning("Warning", "Input is empty!")
            return
        enc_type = self.e_type.get()
        if enc_type == "Base64":
            res = base64.b64decode(t).decode()
        elif enc_type == "Hex":
            res = bytes.fromhex(t).decode()
        else: # URL
            res = urllib.parse.unquote(t)
        self.e_out.delete("1.0", "end")
        self.e_out.insert("1.0", res)
    except Exception as e:
        messagebox.showerror("Error", "Invalid data for decoding. Check your input.")
```

The function "decode" is responsible for decoding encoded text based on the selected encoding type. It first reads the input from the GUI. If the input is empty, it shows a warning message and stops the process.

Then it checks the selected encoding type from "e_type". If the type is **Base64**, it decodes the text using Base64 decoding. If it is **Hex**, it converts the hex string into bytes and then decodes it. If the type is **URL**, it decodes the text using URL decoding.

The result is then displayed in the output textbox. If the input is invalid or an error occurs during decoding, an error message is shown.

Application Entry Point

```
if __name__ == "__main__":
    app = UltimateCryptoToolkit()
    app.mainloop()
```

This block is the starting point of the application.

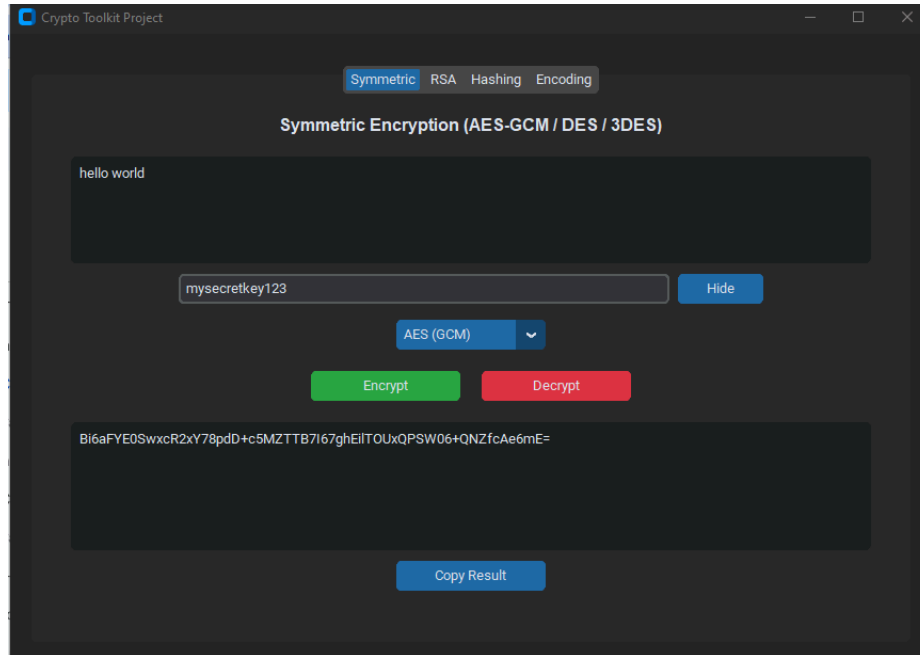
It creates an instance of the UltimateCryptoToolkit class and starts the main event loop using mainloop().

The mainloop keeps the application running and responsive to user interactions such as button clicks and input events.

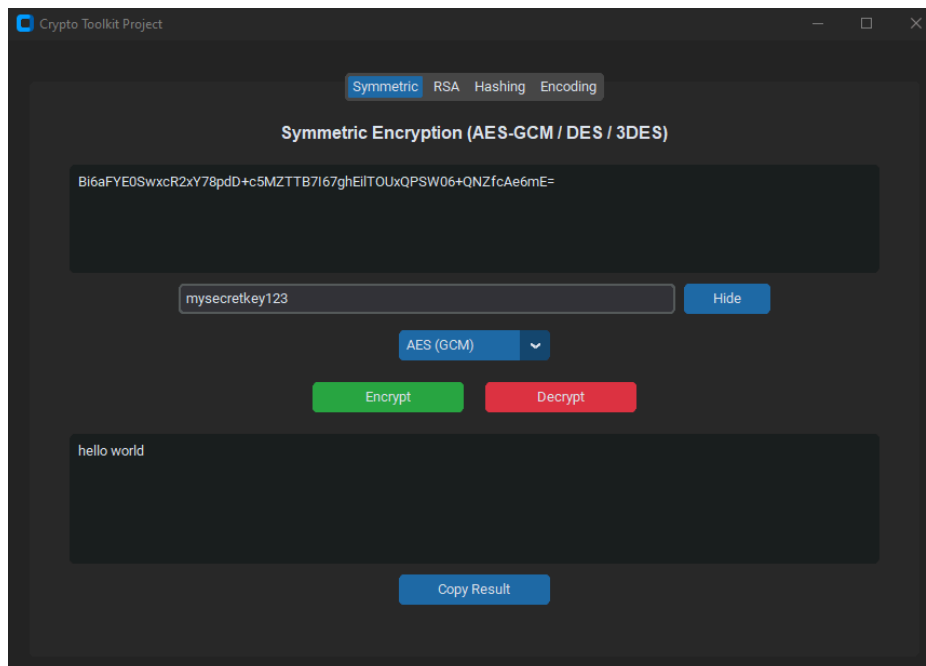
4. Implementation Screenshots

The following screenshots demonstrate the application running each cryptographic feature. Each image shows the GUI with a real input, the selected operation, and the resulting output.

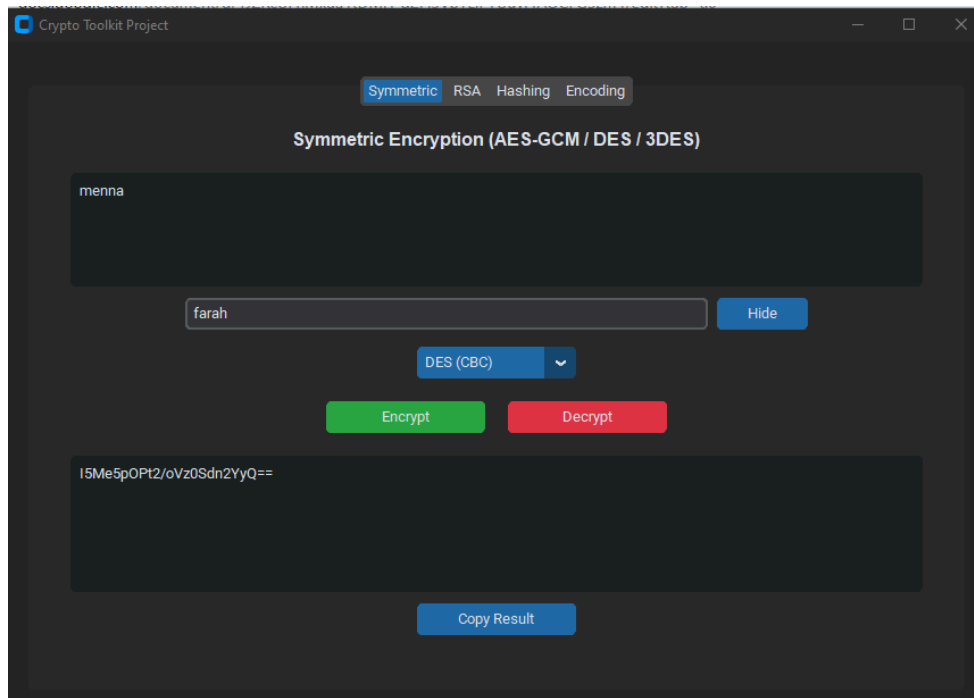
AES Encryption



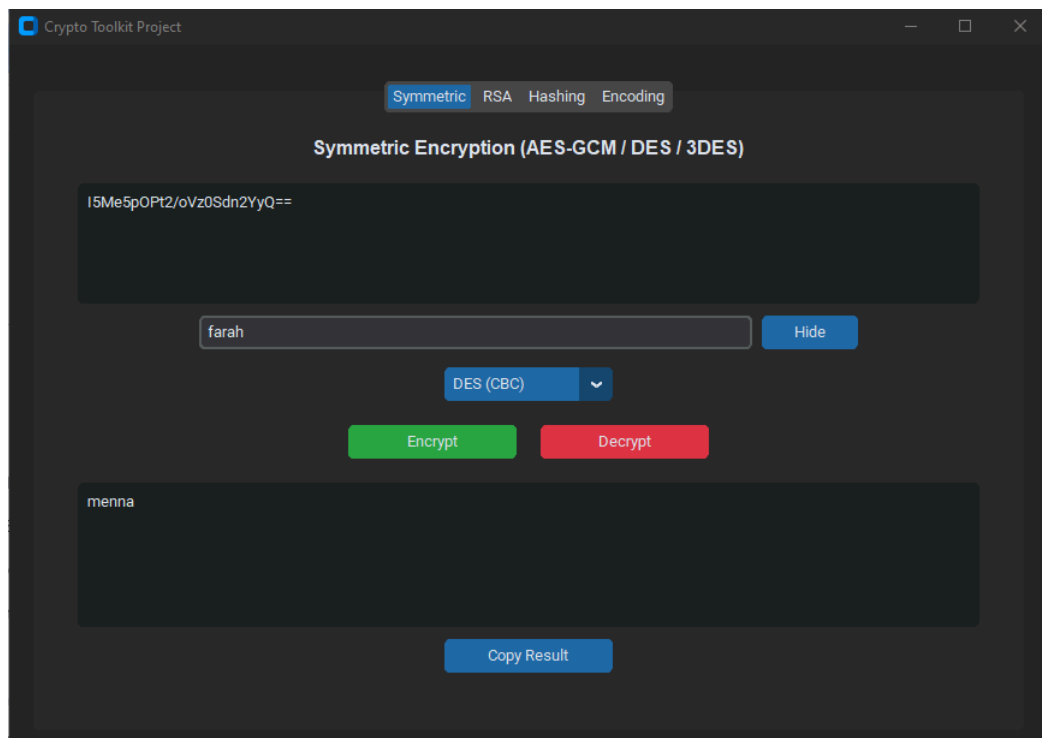
AES Decryption



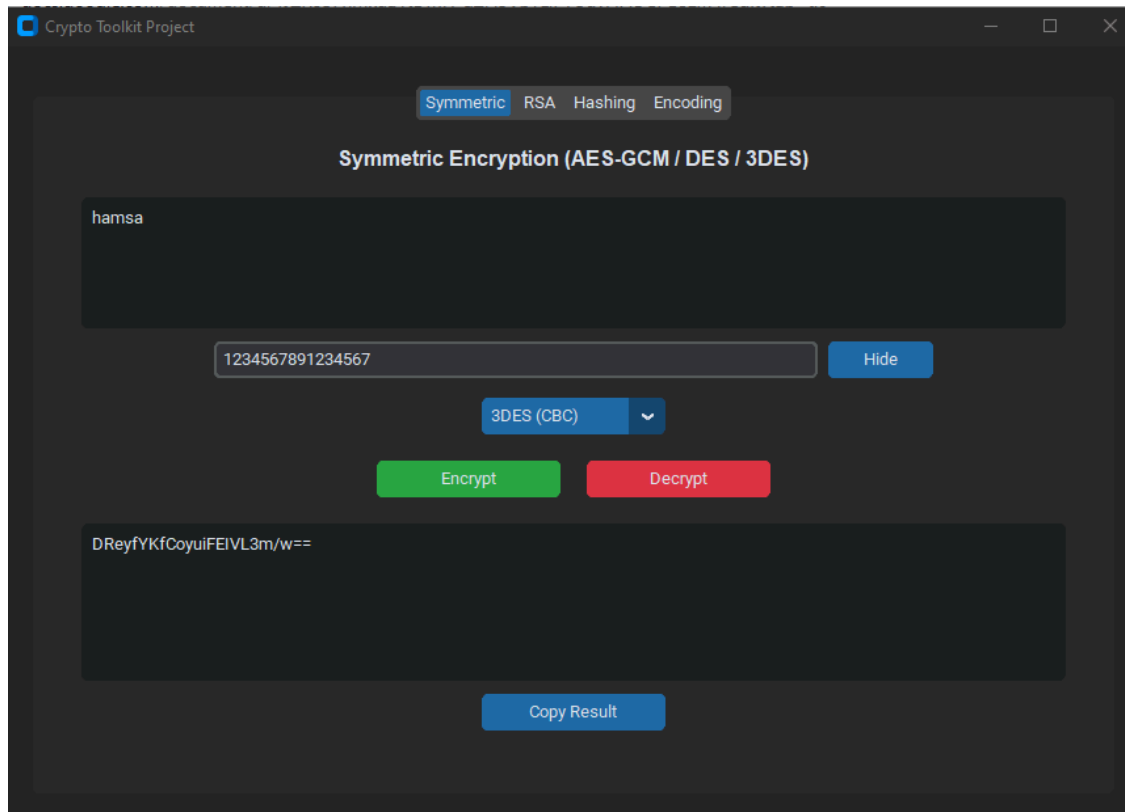
DES Encryption



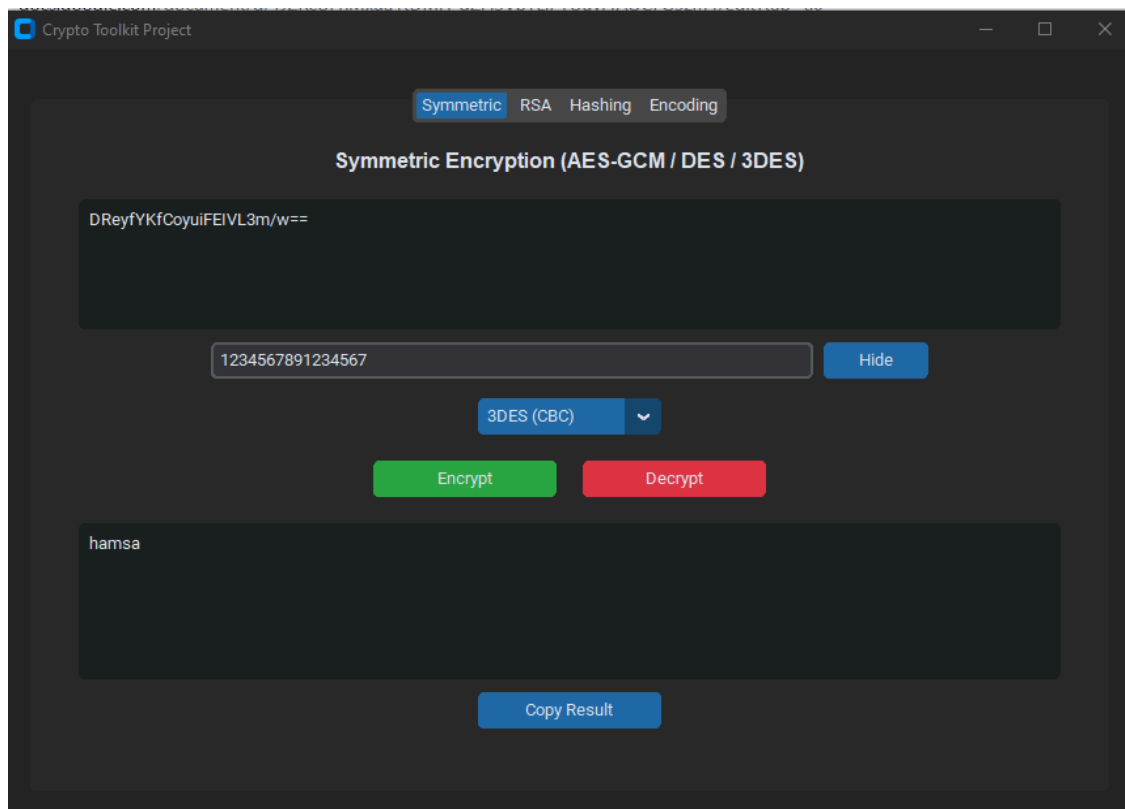
DES Decryption



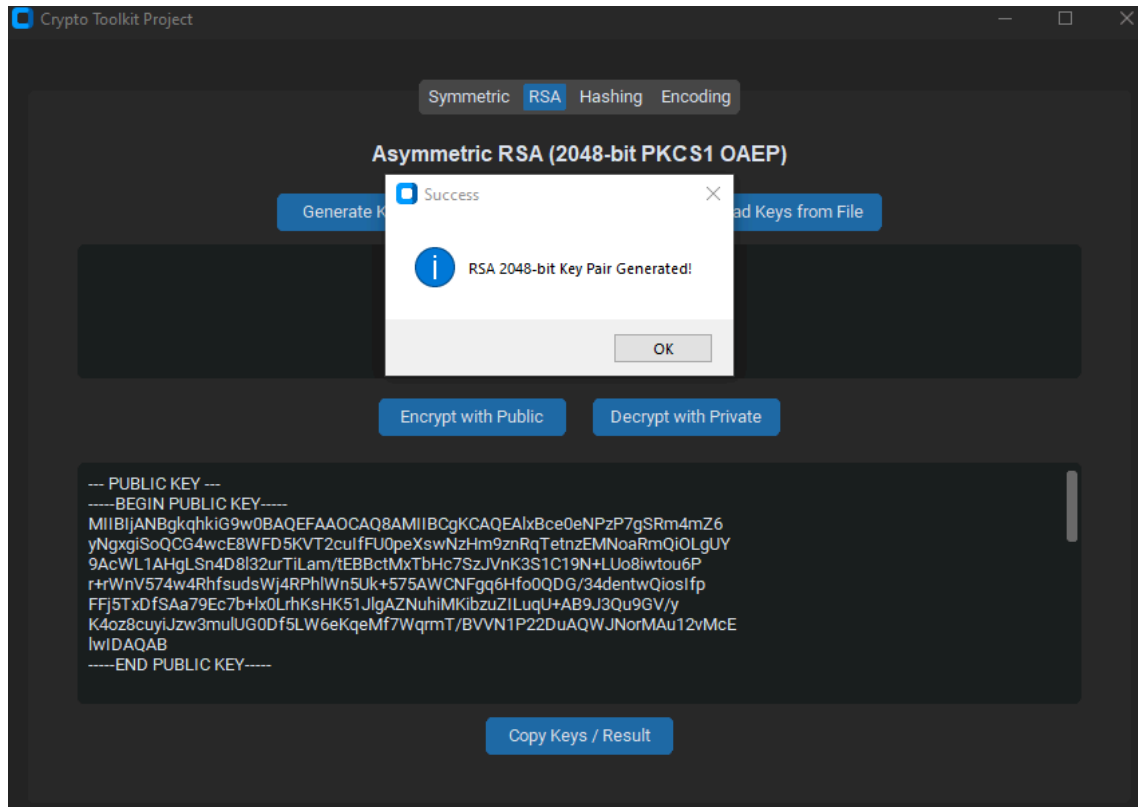
3DES Encryption



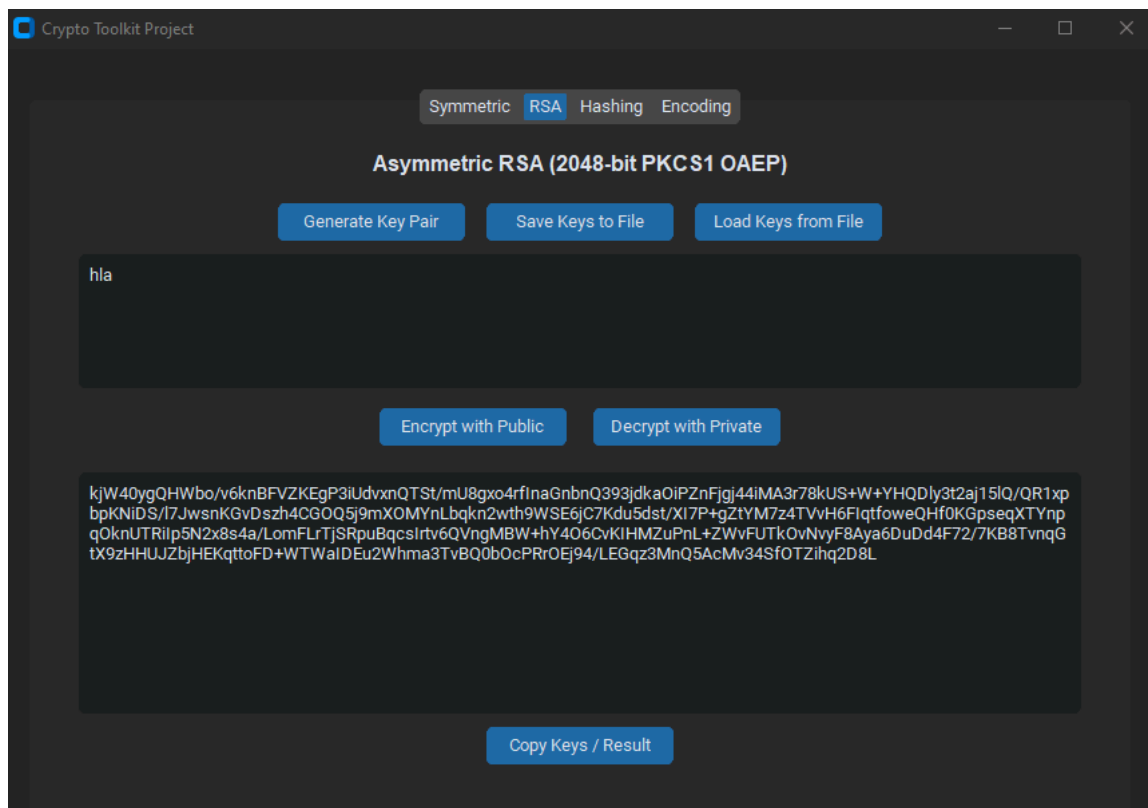
3DES Decryption

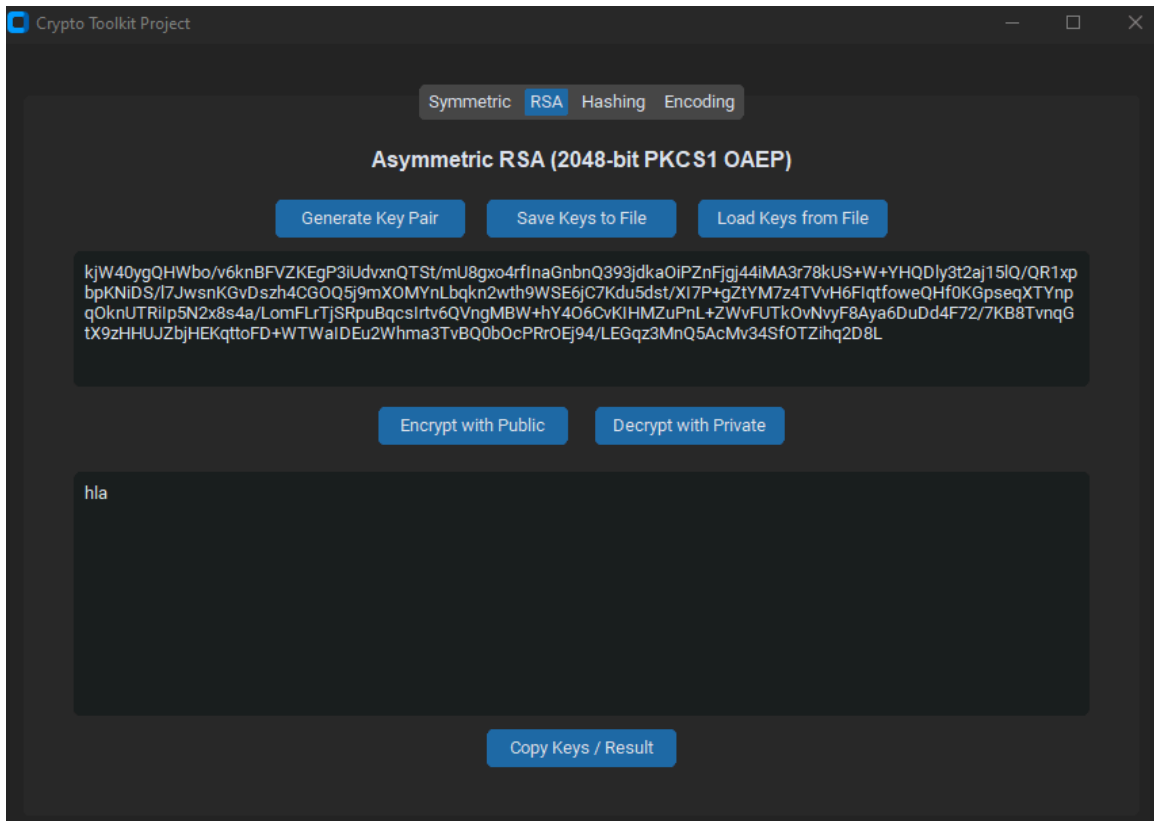


RSA Key Generation

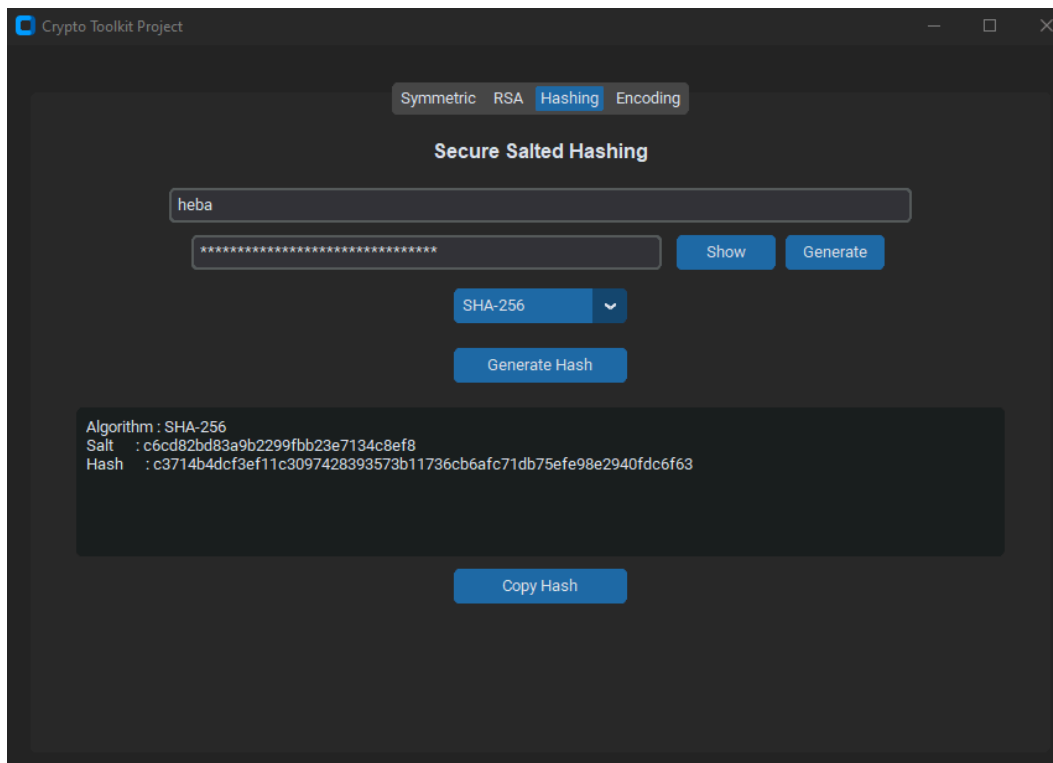


RSA Encryption and Decryption





Hashing



The screenshot shows the 'Hashing' tab of the 'Crypto Toolkit Project' application. The title 'Secure Salted Hashing' is centered. Below it, a text input field contains 'noni'. To its right is a password field with masked characters and 'Show' and 'Generate' buttons. Below these is a dropdown menu set to 'SHA-512' and a 'Generate Hash' button. A results box displays the following information:

```
Algorithm : SHA-512
Salt      : c6cd82bd83a9b2299fbb23e7134c8ef8
Hash      : ddef fd35b929811ca62124448671b4cf998261be82d2ba5727493fd40ec12d0eee76dac0670cb564c7af9606d4aab3d76a2652deb8738d39c814f85cf4ce670c
```

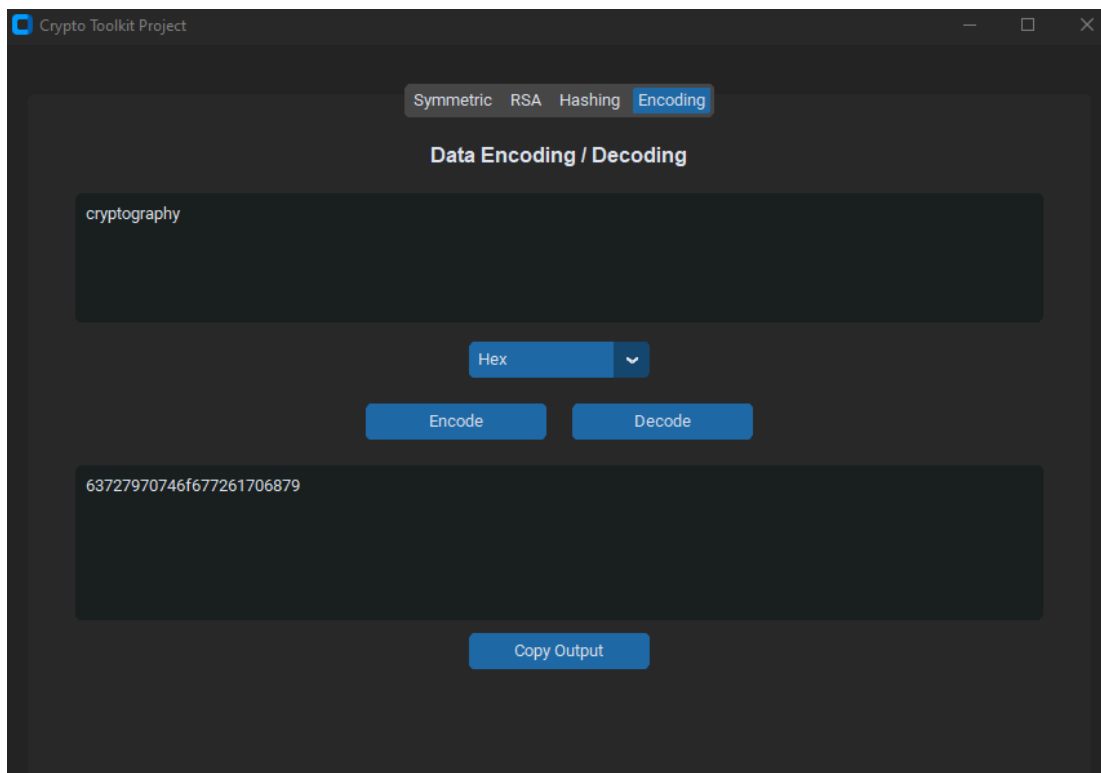
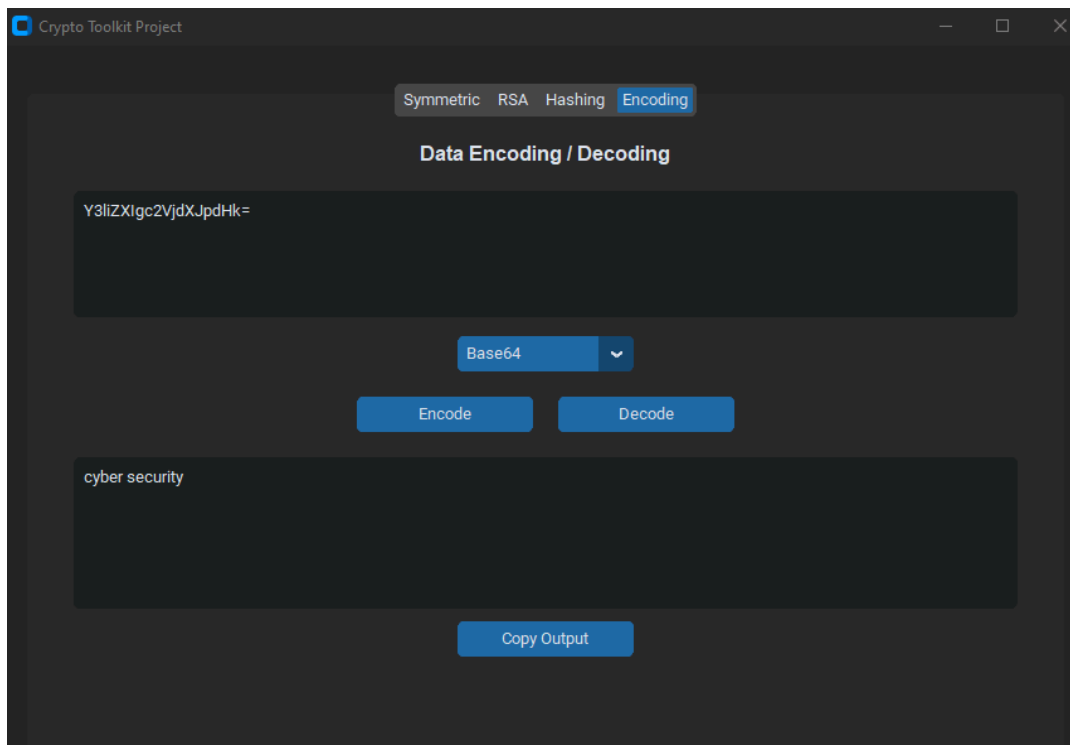
Below the results box is a 'Copy Hash' button.

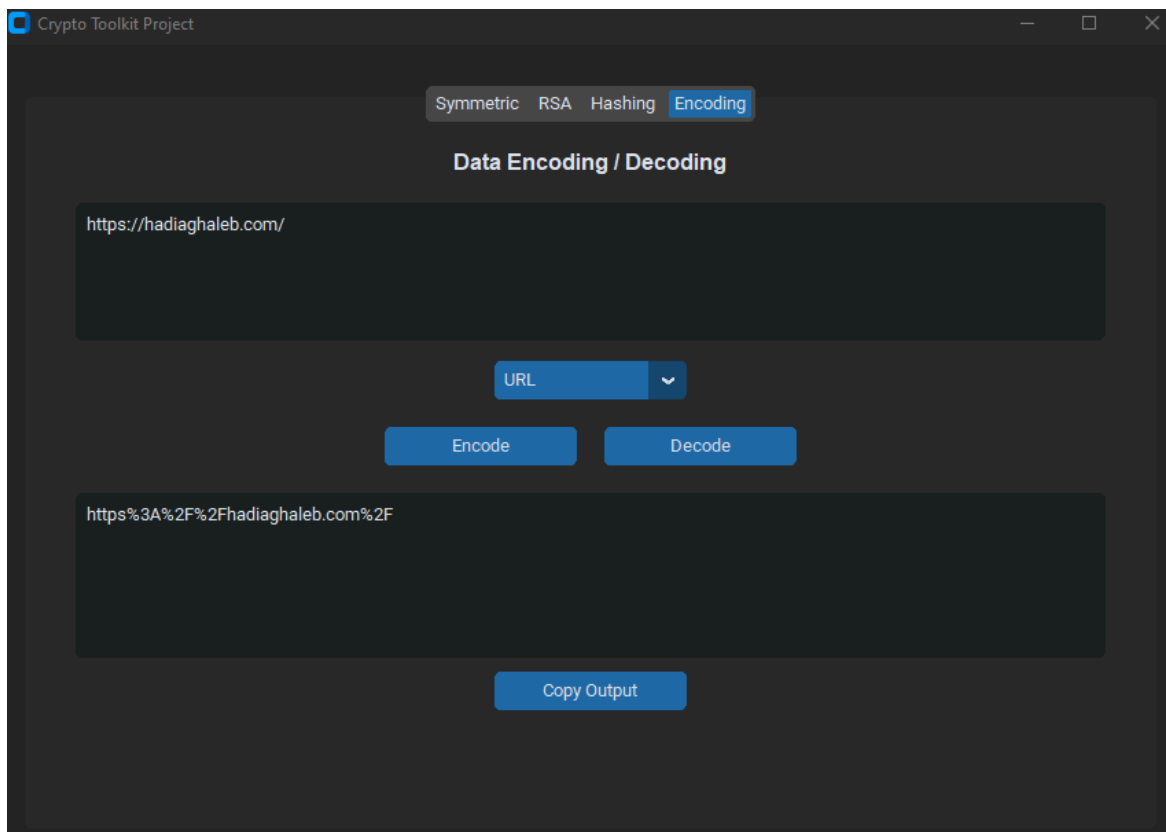
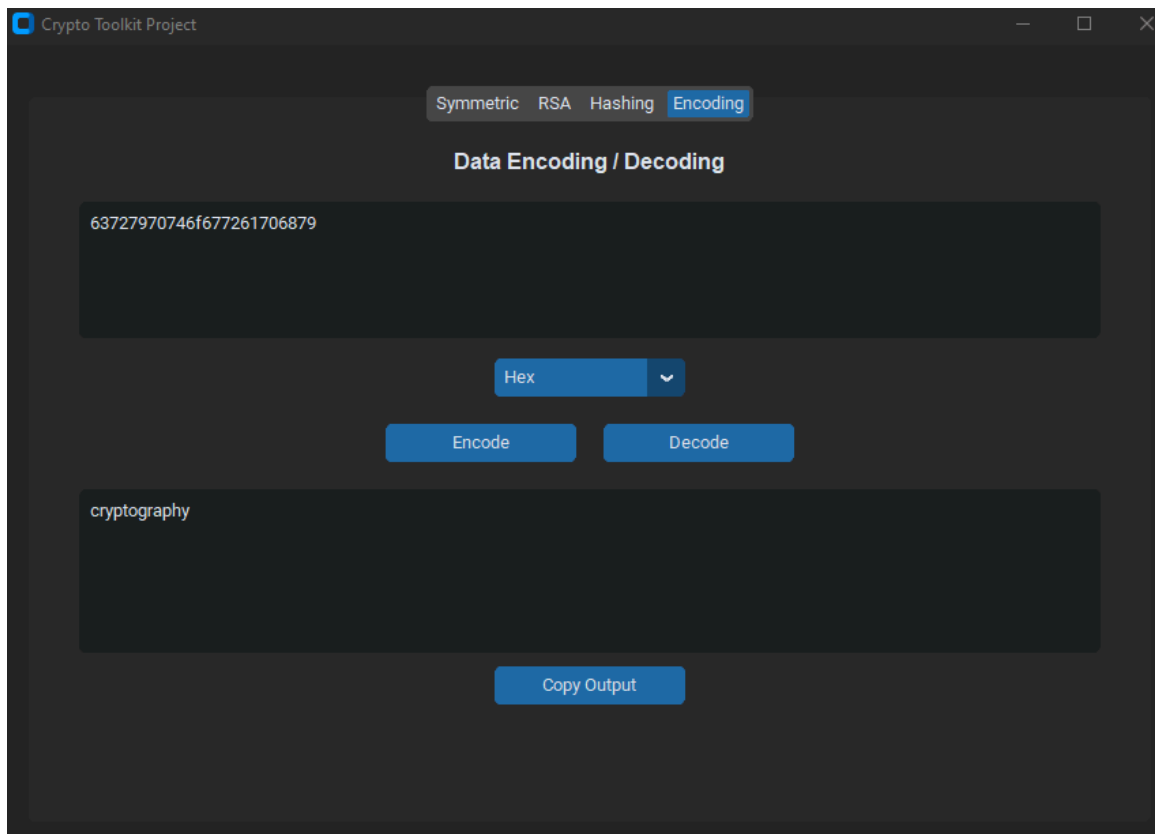
Encoding and Decoding

The screenshot shows the 'Encoding' tab of the 'Crypto Toolkit Project' application. The title 'Data Encoding / Decoding' is centered. Below it, a text input field contains 'cyber security'. To its right is a dropdown menu set to 'Base64' and 'Encode' and 'Decode' buttons. Below these is a results box displaying the encoded output:

```
Y3liZXlgc2VjdXJpdHk=
```

Below the results box is a 'Copy Output' button.





7. Conclusion

The Ultimate Crypto Toolkit project successfully implemented multiple cybersecurity functions in one application, including symmetric encryption, RSA encryption, hashing, and data encoding. The project combined security concepts with a simple graphical interface to make cryptographic operations easier to use and understand.

Through this project, practical knowledge of modern encryption techniques such as AES GCM and RSA OAEP was gained, in addition to secure hashing using SHA algorithms. The project also improved skills in Python programming, GUI development, and secure application designs.

Overall, the project demonstrates how cryptography can be applied in real world applications while maintaining both usability and security.